

Resumen P1 - Fundamentos teóricos

⚙ Estado	Sin empezar
----------	-------------

Parcial 1 - Fundamentos teóricos

Resumen Primer Parcial de Ingeniería de Software

Ejemplo integrador del parcial: Plataforma de cupos de traslados

Descripción general del producto

Problema que resuelve

Actores principales

Usuarios

Transportistas independientes

Administradores de la plataforma

Valor del producto

Unidad 1. Introducción a la Ingeniería del Software

Introducción a la Ingeniería del Software

Software

Diferencia entre programar e ingenierizar software

El software como sistema sociotécnico

Problemas clásicos del desarrollo de software

Naturaleza del software

Ingeniería del Software como disciplina

Definición según Parnas

SWEBOK (TIENE ALGO Q VER CON PARNAS)

SEBok

Integración de todo lo que vimos

Áreas de la Ingeniería de Software

Disciplinas técnicas

Disciplinas de gestión

Disciplinas de soporte

Proceso de software

Procesos definidos vs procesos empíricos

Procesos definidos

Procesos Empíricos

Ciclo de vida

Diferencia entre el ciclo de vida y el ciclo del producto

Ciclo de vida del proyecto

Ciclo de vida del producto

Ciclo de vida y proceso de desarrollo

Clasificación de los ciclos de vida

Modelo secuencial o cascada

Modelo iterativo incremental

Prototipado evolutivo

Modelo en espiral

Ciclo de vida, planificación y decisión estratégica

Ventajas y desventajas de los ciclos de vida

Modelo secuencial

Modelo iterativo incremental

Prototipado evolutivo

Espiral

Criterios para elección de ciclos de vida

En función del proyecto

En función del producto

Ejemplo 

Componentes de un Proyecto de Sistemas de Información

Características de un proyecto

Restricción triple

Relación entre las 3 variables

Equipo de proyecto

Plan de software

Alcance

Estimaciones

Gestión de riesgos

Métricas de software

Tipos de métricas

Métricas básicas de un proyecto

Vínculo proceso-proyecto-producto

Relación conceptual

Ejemplo 

Unidad 2. Gestión de Producto, Requerimientos Ágiles, User Stories, Estimaciones Ágiles y Scrum

Gestión de Producto

Por qué se crean productos

Valor vs desperdicio

El Problema del Sobre-desarrollo

Lean Startup y aprendizaje validado

Supuestos de salto de fe

Primera iteración

MVP

Propuesta de Valor (UVP)

La Audacia del Cero y el Costo del Retraso

Pívor y Adaptación

Ejemplo 📌📌📌

Reducción del riesgo

Ecosistema de los mínimos

Proceso de desarrollo del producto

Experiencia del usuario y evolución del enfoque

Producto Mínimo Adorable (MLP)

Dimensiones del MLP

Ejemplo 📌📌📌

Requerimientos ágiles

Procesos empíricos y requerimientos

Relación entre empírico y agilismo

Agilismo

Valores del agilismo

Individuos e interacciones sobre procesos y herramientas

Software funcionando sobre documentación extensiva

Colaborar con el cliente sobre negociación contractual

Responder al cambio sobre seguir un plan

Principios del Manifiesto Ágil

Requerimientos ágiles

Requerimientos en Agiles

Costo del BRUF

Gestión ágil de requerimientos

Analice cuando lo necesite, no antes

Riqueza del canal de comunicación

Tipos de requerimientos

Dominio del problema y dominio de la solución

Ejemplo 📌📌📌

User Stories

Qué son

Product Backlog y Product Owner

Historias como porciones verticales

Partes de las User Stories

1. Estructura básica

2. Modelado de roles

3. Criterios de aceptación

Para qué sirven

Cómo debe ser un buen criterio de aceptación

¿Dónde van los detalles?

4. Pruebas de aceptación

Relación con los criterios

Proceso de dos pasos

Ejemplo 

Ejemplo 

Definition of Ready

Definition of Done

INVEST

Spikes

Ejemplo 

Estimaciones tradicionales

Para qué estimamos

Error de estimación

Técnicas fundamentales de estimación

Métodos de estimación

Datos históricos

Juicio experto puro

Juicio experto estructurado

Wideband Delphi

Estimaciones ágiles

Estimación relativa

Tamaño de una historia

Tamaño vs esfuerzo

Escalas para estimar tamaño

Story points

Velocidad

Planning Poker

Ejemplo 

Framework Scrum

Qué es Scrum

Roles

Product Owner

Scrum Master

Developers

Artefactos

Product Backlog

Sprint Backlog

Incremento

Eventos

Sprint

Sprint Planning

Daily Scrum

Sprint Review

Sprint Retrospective

Relación con requerimientos y estimaciones

Aplicación al ejemplo integrador

Unidad 3. Gestión de Configuración de Software (SCM)

El software en contexto

Por qué cambia el software

Relación con otras disciplinas

Problemas sin un buen SCM

Conceptos clave de SCM

Línea base

Ramas

Actividades fundamentales de SCM

Identificación de ítems

Control de cambios

Comité de Control de Cambios

Auditorías de configuración

Auditoría física (PCA)

Auditoría funcional (FCA)

Relación con verificación y validación

Informes de estado

Plan de SCM

SCM y desarrollo ágil

Ejemplo 

Parcial 1 - Fundamentos teóricos

Resumen Primer Parcial de Ingeniería de Software

Ejemplo integrador del parcial: Plataforma de cupos de traslados

Descripción general del producto

La **plataforma de cupos de traslados** es un producto de software de intermediación digital que conecta a dos lados de un mercado: por un lado, usuarios que necesitan conseguir un traslado y, por otro, transportistas independientes que ofrecen lugares disponibles. La lógica general se parece a la de un marketplace porque la plataforma no presta directamente el servicio

de transporte, sino que **hace visible la oferta**, facilita el descubrimiento de opciones, organiza la interacción entre las partes y agrega mecanismos que reducen fricción, incertidumbre y costos de coordinación.

Problema que resuelve

Antes de una plataforma de este tipo, la coordinación de cupos suele hacerse a través de contactos informales, publicaciones dispersas, mensajes privados, grupos, llamadas o recomendaciones personales. Ese esquema genera varios problemas: poca visibilidad de la oferta real, baja ocupación de plazas, dificultad para comparar opciones, escasa trazabilidad, mayor tiempo de coordinación y menor previsibilidad para usuarios y transportistas. La plataforma busca transformar ese problema desordenado en un entorno digital estructurado, donde la información sea accesible, comparable y reusable.

Actores principales

Usuarios

Son quienes necesitan un traslado. Su expectativa principal es encontrar una opción conveniente, confiable y clara. Desde la perspectiva del producto, valoran velocidad de búsqueda, disponibilidad de información, facilidad de coordinación y confianza en el sistema.

Transportistas independientes

Son quienes publican cupos y gestionan disponibilidad. Para ellos, el valor del producto está en la visibilidad, el acceso a potenciales pasajeros, la reducción de tiempos muertos y la mejora en la ocupación de plazas.

Administradores de la plataforma

Supervisan el ecosistema, gestionan reglas del negocio, atienden incidencias, resguardan la coherencia del sistema y participan en su evolución.

Valor del producto

El valor del producto no se reduce a “tener una app que funcione”. Desde la lógica de gestión de producto, el valor existe cuando la solución modifica positivamente una situación real del usuario y además resulta sostenible para el negocio. En este caso:

- para el usuario, el valor está en **encontrar, comparar y coordinar** traslados de forma más clara y rápida;
 - para el transportista, el valor está en **exponer su oferta** y mejorar la ocupación de cupos;
 - para el negocio, el valor está en **construir una plataforma escalable** basada en interacción recurrente entre oferta y demanda.
-

Unidad 1. Introducción a la Ingeniería del Software

Introducción a la Ingeniería del Software

Software

Software ■ Conjunto de programas que vienen junto a su documentación y que son intangibles, abstractos y no restringido por leyes físicas

Hay que tener en cuenta que el software involucra también todo lo que lo rodea, ya que cuando entregamos software, no entregamos solo código, sino un sistema completo que debe funcionar, mantenerse y evolucionar

Diferencia entre programar e ingenierizar software



Programar es una actividad central, pero la Ingeniería del Software incorpora además proceso, proyecto, producto, calidad y gestión.

Programar es construir instrucciones ejecutables. Ingenierizar software implica algo mucho más amplio:

1. Comprender un problema
2. Modelar el problema
3. Diseñar una solución
4. Organizar el proceso de construcción
5. Validar resultados
6. Gestionar cambios

7. Medir calidad
8. Sostener el producto durante su evolución.

El software como sistema sociotécnico



El software no debe pensarse solo como código, sino como parte de un sistema más amplio que involucra personas, reglas, procesos de trabajo, datos, infraestructura y objetivos de negocio.

Software en ISW 📖 Es un sistema sociotécnico y que al entregar software no se entrega solo código, sino un sistema completo que debe funcionar, mantenerse y evolucionar.

Esto evita una visión reducida del problema.

Ejemplo 📌📌📌

En la plataforma de cupos de traslados, por ejemplo, el sistema no se agota en una base de datos y una interfaz. También incluye:

- reglas de publicación de cupos;
- comportamiento de usuarios y transportistas;
- expectativas de confianza;
- políticas de disponibilidad;
- flujos de comunicación;
- eventuales normas regulatorias;
- necesidades de soporte y evolución.



Decir que el software es sociotécnico, se está diciendo que su funcionamiento real depende de la articulación entre tecnología, personas y procesos

Problemas clásicos del desarrollo de software

Tus resúmenes destacan varios problemas clásicos que siguen siendo centrales:

1. La versión final no satisface al cliente

El problema no suele ser técnico, sino de entendimiento. El sistema puede estar bien programado, pero no resolver lo que el cliente realmente necesita.

- Muchos errores vienen de la etapa de requerimientos
- No hubo comunicación efectiva
- El cliente cambio necesidades

2. No es fácil extenderlo o adaptarlo

Si el software no está bien diseñado, cada cambio se vuelve costoso o directamente imposible. Por eso la arquitectura es clave.

- El software debe diseñarse pensando en el cambio
- El software siempre evoluciona
- Hay que tener en cuenta que el software que no puede cambiar, muere

3. Mala documentación

La documentación es lo que permite que el software sobreviva en el tiempo. Sin ella, el sistema depende de quienes lo hicieron.

- El software debe incluir documentación, no es opcional
- Si no tiene software puede traer problemas de mantenimiento, de no entender y de dependencia

4. Mala calidad

La calidad del software no es solo que no tenga errores, sino que sea usable, eficiente y confiable.

- La calidad no es solo que funcione, sino como funciona
- Que funcione no significa que sea bueno

5. Mas tiempo y costo de lo presupuestado

El software rara vez cumple los tiempos estimados porque es difícil anticipar todos los problemas que van a surgir

- Estimar software es difícil por su naturaleza incierta
 - Por eso se dice que en software, estimar es aproximar, no es asegurar

Todos estos problemas fueron los que dieron origen a la ingeniería de software como disciplina. Justamente surge para ordenar, estructurar y mejorar el desarrollo de sistemas.



Los principales problemas no están en el código, sino en los requerimientos, la comunicación y la gestión.

Se necesita aplicar principios de ingeniería de software para poder construir sistemas que realmente funcionen, evolucionen y satisfagan al usuario

Naturaleza del software



El software se caracteriza como un conjunto de **programas + datos + documentación**. Entonces:

El software no es solo el ejecutable sino también son todos los artefactos que permiten construirlo, entenderlo, operarlo y modificarlo

1. El software es menos predecible

Manufactura → Los procesos son repetitivos y pueden estimarse

Software → Cada desarrollo implica incertidumbre

El software no es predecible porque depende de decisiones humanas, cambios de contexto y problemas nuevos que aparecen durante el desarrollo. Por eso no podemos planificarlo como una línea de producción.

2. No hay producción en masa

Manufactura → Se producen miles de unidades iguales

Software → Cada sistema es único y cada versión cambia el producto

Aunque usemos el mismo código base, cada software evoluciona de forma distinta. No existe una producción en serie como en la industria

3. No todas las fallas son errores



Verificación: Esta bien construido ?
Validación: Es lo correcto ?

Manufactura → Defecto técnico

Software → Pueden fallar porque el usuario cambio de idea o el contexto cambio

En software, una falla no siempre es un error técnico. Muchas veces el problema está en lo que se pidió, no en cómo se programó.

4. El software no se gasta

Manufactura → Los productos se desgastan con el uso

Software → Hay degradacion logica ya que mientras el software evoluciona, la complejidad aumenta

El software no se rompe por uso, pero se vuelve más complejo con cada cambio. Es un deterioro lógico, no físico

5. El software no está gobernado por leyes de la física

Ingeniería del Software como disciplina

Definición según Parnas

ISW según Parnas ■ La Ingeniería del Software es la construcción de software por múltiples personas y en múltiples versiones. Este termino maneja estas dos características:

- la coordinación entre personas.
- la evolución del producto a través del tiempo.

Es una actividad colaborativa y evolutiva que exige procesos, estructuras y mecanismos de control.

SWEBOK (TIENE ALGO Q VER CON PARNAS)

SWEBOK ■ Cuerpo de conocimiento de la ingeniería de software

El SWEBOK es el cuerpo de conocimiento de la ingeniería de software definido por la IEEE.

Organiza la disciplina en distintas áreas como requerimientos, diseño, construcción, testing y mantenimiento, lo que demuestra que la ingeniería de software es una disciplina estructurada y no simplemente programación

SEBok



SWEBOK → software

SEBoK → sistema completo

El SEBoK organiza el conocimiento de la ingeniería de sistemas, donde el software es solo una parte de un sistema más amplio que incluye personas, procesos y tecnología.

Esto es importante porque en la práctica los sistemas reales son complejos y requieren una visión integral

Tiene 5 partes:

1. Fundamentos de la ingeniería de sistemas
2. Ingeniería de Sistemas y gestión
3. Aplicaciones de la ingeniería de sistemas
4. Facilitadores de la ingeniería de sistemas
5. Disciplinas relacionadas

Integración de todo lo que vimos

El software es un conjunto de programas, datos y documentación que permiten resolver problemas en el mundo real.

Sin embargo, el desarrollo de software presenta muchos problemas, como fallas, requerimientos mal definidos y dificultad para adaptarse a cambios.

Esto ocurre porque el software no puede tratarse como un proceso de manufactura, ya que no es repetitivo ni predecible, sino que implica incertidumbre y evolución constante.

Debido a esto, surge la ingeniería de software como disciplina, con el objetivo de organizar y mejorar el desarrollo de sistemas.

Luego, Parnas plantea que el software es un proceso colaborativo y evolutivo, ya que involucra múltiples personas y múltiples versiones.

Finalmente, la IEEE organiza la ingeniería de software a través del SWEBOK, definiendo distintas áreas de conocimiento como requerimientos, diseño,

testing y mantenimiento, lo que demuestra que no se trata solo de programar, sino de una disciplina completa.



Software = programas + datos + documentación

Problema = falla, cambio, incertidumbre

No es manufactura = no es predecible

ISW = solución para organizar

Parnas = equipo + versiones

SWEBOK = áreas de conocimiento

SEBoK = sistema completo

Áreas de la Ingeniería de Software

La Ingeniería de Software se estructura en un conjunto de disciplinas que permiten abordar el desarrollo de sistemas de manera organizada, controlada y orientada a la calidad.

Disciplinas técnicas

Técnicas Las disciplinas técnicas constituyen el núcleo del desarrollo, ya que se encargan directamente de la construcción del software.

1. Requerimientos

La disciplina de requerimientos tiene como objetivo identificar, analizar y documentar las necesidades del usuario o cliente.

En esta etapa se define **qué debe hacer el sistema**, estableciendo tanto los requisitos funcionales (acciones que el sistema debe realizar) como los no funcionales (rendimiento, seguridad, usabilidad, entre otros).

2. Análisis y diseño

Una vez definidos los requerimientos, se procede a analizar el problema y diseñar una solución técnica.

El análisis implica comprender en profundidad el dominio del problema, mientras que el diseño establece la estructura del sistema, incluyendo su arquitectura, componentes, relaciones y comportamiento.

- Es la traducción de necesidades en una solución estructurada

3. Construcción

La construcción corresponde a la implementación del sistema, es decir, la codificación del software utilizando lenguajes de programación y herramientas tecnológicas.

En esta etapa se materializa el diseño en código ejecutable. Sin embargo, no se trata solo de programar, sino de aplicar buenas prácticas, patrones de diseño y estándares que permitan generar un software mantenible y escalable.

4. Prueba

La disciplina de prueba tiene como objetivo verificar y validar que el software cumple con los requerimientos definidos.

Se realizan distintos tipos de pruebas, como:

- pruebas unitarias
- pruebas de integración
- pruebas de sistema
- pruebas de aceptación

5. Despliegue

El despliegue consiste en la puesta en producción del sistema, es decir, hacerlo disponible para su uso real.

Incluye actividades como:

- configuración de servidores
- integración con servicios externos
- publicación del sistema

También puede involucrar estrategias como despliegue continuo (CI/CD), actualizaciones progresivas y monitoreo inicial.

Disciplinas de gestión

Gestión Las disciplinas de gestión se encargan de organizar, planificar y controlar el desarrollo del software, asegurando que el proyecto avance de manera ordenada y cumpla con los objetivos establecidos.

1. Planificación de proyecto

La planificación de proyecto tiene como objetivo definir cómo se llevará a cabo el desarrollo del software.

En esta etapa se establecen:

- los tiempos del proyecto
- los recursos necesarios
- las tareas a realizar
- los responsables de cada actividad

2. Monitoreo y control de proyectos

El monitoreo y control se encarga de supervisar el avance del proyecto y verificar que se cumpla lo planificado.

En esta etapa se realiza:

- seguimiento de tareas
- control de tiempos y costos
- detección de desvíos
- gestión de riesgos

Permite tomar decisiones correctivas en caso de que el proyecto no avance según lo previsto.

Disciplinas de soporte

Soporte Las disciplinas de soporte tienen como objetivo garantizar la calidad del software, su mantenimiento en el tiempo y su correcta evolución.

1. Gestión de configuración de software

La gestión de configuración se encarga de controlar los cambios que se realizan sobre el software a lo largo de su ciclo de vida.

Incluye:

- control de versiones
- registro de cambios
- gestión de código fuente
- administración de distintas versiones del sistema

2. Aseguramiento de calidad

El aseguramiento de calidad tiene como objetivo garantizar que el software cumpla con estándares definidos y satisfaga las necesidades del usuario.

No se limita a la etapa de pruebas, sino que abarca todo el proceso de desarrollo:

- definición de estándares
- revisiones de código
- control de procesos
- mejora continua

Busca prevenir errores antes de que ocurran.

3. Métricas

Las métricas permiten medir distintos aspectos del software y del proceso de desarrollo.

Se utilizan para evaluar:

- calidad del software
- rendimiento del sistema
- productividad del equipo
- cumplimiento de plazos

Estas mediciones permiten tomar decisiones basadas en datos y mejorar continuamente el proceso.

- Es la base para la mejora continua.

Proceso de software

Proceso de software  Conjunto estructurado de actividades, métodos, practicas y transformaciones que se usan para desarrollar y mantener un sistema de software.

- Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse

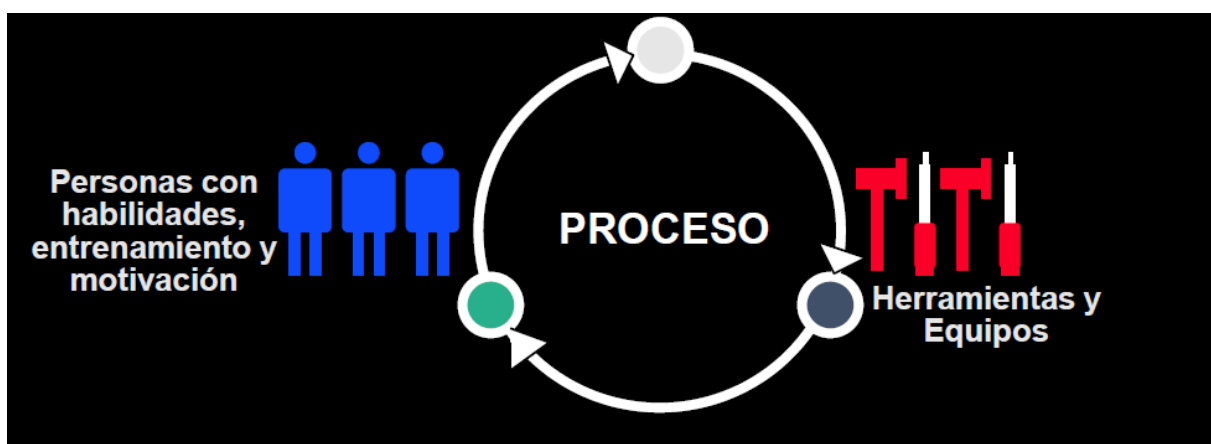


Transformar necesidades en soluciones mediante un proceso organizado

El proceso de software comienza con entradas como requerimientos, ideas y tiempo, que definen qué se debe construir.

Luego, se utilizan recursos como personas, herramientas y procedimientos para llevar a cabo actividades de trabajo, donde se analiza, diseña, construye y prueba el sistema.

Finalmente, el proceso genera como salida un producto o servicio de software que aporta valor al usuario



El proceso de software se representa como un ciclo iterativo, no lineal, ya que el sistema evoluciona constantemente.

Este proceso no depende solo de actividades técnicas, sino también de tres elementos fundamentales: las personas, las herramientas y los métodos de trabajo.

Las personas aportan conocimiento y toma de decisiones, las herramientas facilitan el desarrollo y los métodos organizan el trabajo.

La interacción de estos elementos permite desarrollar software de manera continua y controlada.

Procesos definidos vs procesos empíricos

Procesos definidos

Proceso definido ■ Las actividades, tareas, secuencias y resultados están previamente especificados y documentados.

Asume que podemos repetir el mismo proceso una y otra vez, indefinidamente, y obtener los mismos resultados

Pressman dice:

- Un proceso definido es predecible
- Se basa en una secuencia estructurada de actividades
- Permite planificar, controlar y medir el desarrollo

Sommerville dice:

- Algunos procesos intentan modelar completamente el desarrollo
- Se basan en la idea de que los requerimientos pueden definirse al inicio y que además el sistema puede diseñarse completamente antes de implementarse

Se basan en la idea de que el desarrollo es predecible y repetible, permitiendo planificar y controlar el proyecto mediante una secuencia estructurada de fases.

Sin embargo, en el desarrollo de software estos supuestos no siempre se cumplen debido a la incertidumbre y los cambios constantes en los requerimientos, lo que limita su aplicabilidad en contextos complejos.

Procesos Empíricos

Procesos empíricos ■ El conocimiento del sistema se obtiene a medida que se desarrolla

- El concepto de procesos empíricos provienen de sistemas complejos y creativos donde no hay soluciones únicas y no se pueden predecir todo

Sommerville dice:

- En muchos sistemas no se pueden definir completamente los requerimientos al inicio
- El desarrollo debe ser iterativo e incremental
- Se aprende del sistema mientras se construye

Pressman dice:

- El desarrollo de software esta fuertemente influenciado por:
 - Factores humanos
 - Incertidumbre
 - Cambios constantes
- Por ende en base a esto, no pueden ser completamente predecibles

Ciclo empírico

1. Asumir
2. Construir
3. Retroalimentar
4. Revisar
5. Adaptar

Se basan en la iteración, el feedback constante y la adaptación, ya que no es posible definir completamente el sistema al inicio.

Este enfoque es especialmente adecuado para el desarrollo de software, debido a su naturaleza compleja y cambiante.

Ciclo de vida

Ciclo de vida  Un ciclo de vida es la serie de etapas por las que pasa un producto o proyecto desde que nace hasta que se retira, esto aplica a:

- El proyecto, que es el esfuerzo organizado para construir el software
- El producto, que es el software ya existente en uso, evolucionando con el tiempo

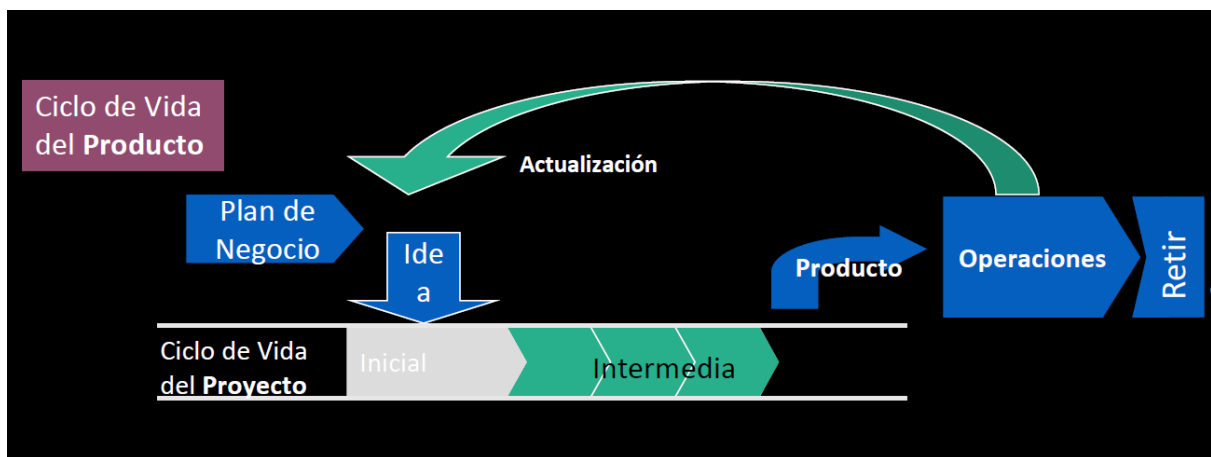
El modelo de ciclo de vida especifica:

1. Las fases del proceso
2. El orden en las que las fases se realizan

Un ciclo de vida define:

1. La forma prescriptiva que debería ocurrir entre el primer destello de idea y la salida del producto
2. El orden de actividades como especificar, prototipar, diseñar, implementar, revisar y probar

Diferencia entre el ciclo de vida y el ciclo del producto



Ciclo de vida del proyecto



El proyecto tiene un comienzo y un fin mas definido.
El objetivo es ENTREGAR ALGO

El **ciclo de vida del proyecto** se refiere al esfuerzo de construcción. Empieza cuando se decide desarrollar el sistema y termina cuando ese esfuerzo concluye, normalmente con una entrega o liberación.

Ciclo de vida del producto

El **ciclo de vida del producto** es más largo. Empieza con la idea y el plan de negocio, sigue con el proyecto que lo construye, continúa con la operación real del sistema y se extiende mediante actualizaciones, mantenimiento y evolución, hasta su retiro.



El proyecto termina antes que el producto

Sommeville dice que la operacion y el mantenimiento es la fase mas larga del ciclo de vida por las nuevas operaciones, necesidades y mejoras que van surgiendo

Ciclo de vida y proceso de desarrollo



Cuando hablan de ciclo de vida, es una muestra de la forma de organizar y visualizar el proceso

Es decir, el proceso define las actividades y el ciclo de vida las organiza en el tiempo

En ISW, los conceptos de ciclo de vida y proceso de desarrollo se complementan

Proceso de desarrollo de software 📌 Se refiere al conjunto de actividades, métodos y practicas necesarias para construir y mantener un sistema

- El proceso de desarrollo responde a la pregunta de que se hace y como se hace
- El proceso de desarrollo abarca el inicio hasta el fin del desarrollo

El modelo de ciclo de vida 📌 Esquema que establece el orden en que un proyecto especifica, diseña, implementa, prueba y revisa el software, funcionando como el plan maestro del proyecto

- Es una representación del proceso desde una perspectiva temporal
- El ciclo de vida responde a la pregunta de cuando se hace cada actividad y en que secuencia

Clasificación de los ciclos de vida

Modelo secuencial o cascada

- Es más rígido, ordenado por fases sucesivas.
- A nivel teórico, la fase no debería comenzar hasta terminar la anterior, pero en la practica, estas fases se superponen y retroalimentan

- Su fortaleza está en la planificación, la trazabilidad y la documentación.
- Su debilidad está en la baja adaptabilidad al cambio.

Modelo iterativo incremental

- Entrega el producto por partes útiles.
- Permite feedback temprano y mejor gestión de incertidumbre
- Existen ciclos de vida como el evolutionary prototyping, staged delivery y evolutionary delivery que se apoyan en este modelo.
- Se relaciona con lo empirico

Prototipado evolutivo

- Se orienta a comprender mejor requerimientos y validar ideas.
- Su gran ventaja es volver visible lo implícito
- Su riesgo es convertir un prototipo improvisado en sistema final.

Modelo en espiral

- Integra desarrollo iterativo con análisis explícito de riesgo.
- Es especialmente valioso cuando hay alta incertidumbre técnica o funcional.
- Se puede iniciar dentro de un ciclo, un subciclo

Ciclo de vida, planificación y decisión estratégica



McConnel dice que:

- Si el proyecto puede mantener un enfoque lineal tipo waterfall, hará mas rapido el desarrollo
- Si el proyecto tiene motivos para no sostener la linealidad, es mejor un enfoque mas flexible

Esto es clave porque no demoniza lo secuencial ni idealiza lo iterativo, lo contextualiza

McConnell dice que el modelo de ciclo de vida define el MASTER PLAN del proyecto

Ademas planeta que no existe un modelo de ciclo de vida mas rapido, sino que en realidad, se basa por el contexto

- el ciclo de vida es una representación temporal del proceso,
- organiza fases y transiciones,
- expresa una visión sobre la estabilidad o inestabilidad de requerimientos,
- define cuánta planificación previa se hace,
- cuánto se tolera el cambio,
- cuánta visibilidad se ofrece,
- y cómo se equilibran velocidad, riesgo, control y adaptación.

Ventajas y desventajas de los ciclos de vida

Modelo secuencial

Ventajas

- Capacidad para facilitar la planificación y el control de proyecto ya que establece una secuencia clara de fases
- Generación de documentación detallada en cada etapa

Desventajas

- baja tolerancia al cambio;
- retrocesos costosos;
- errores descubiertos tarde;
- riesgo de alejarse de la necesidad real.

Modelo iterativo incremental

Ventajas

- validación temprana;
- adaptación al cambio;
- reducción de riesgo por entregas parciales;

- aprendizaje progresivo.

Desventajas

- requiere más coordinación;
- puede debilitar coherencia arquitectónica si se gestiona mal;
- dificulta algunas estimaciones tradicionales.

Prototipado evolutivo

Ventajas

- mejora comprensión;
- reduce ambigüedad inicial;
- favorece participación del usuario.

Desventajas

- puede generar falsa sensación de avance;
- riesgo estructural si no se rediseña.

Espiral

Ventajas

- gestión explícita del riesgo;
- combinación de estructura y adaptación.

Desventajas

- alta complejidad de gestión;
- costo elevado para proyectos simples.

Criterios para elección de ciclos de vida

En función del proyecto

Influyen especialmente:

- claridad o inestabilidad de requerimientos;
- nivel de riesgo;
- necesidad de feedback continuo;

- tamaño y madurez del equipo;
- exigencias de control y documentación.

En función del producto

También pesan:

- complejidad del sistema;
- criticidad y confiabilidad requerida;
- importancia de la experiencia de usuario;
- frecuencia esperada de cambio.

Ejemplo

Para la plataforma de cupos de traslados, un ciclo de vida iterativo incremental parece el más razonable, porque:

- el valor debe validarse con usuarios reales;
- la experiencia de uso será importante;
- el negocio probablemente cambie con el aprendizaje;
- conviene lanzar versiones funcionales temprano.

Componentes de un Proyecto de Sistemas de Información

Componente de proyecto ¶ Parte estructurada del proyecto de software que agrupa actividades, tareas y entregables relacionados, dentro de un proceso de desarrollo

Proyecto ¶ Un esfuerzo temporal organizado que tiene como objetivo producir un resultado único, mediante la ejecución coordinada de actividades y el uso de recursos limitados.

- Un proyecto es el contexto donde se aplica el proceso de software

Características de un proyecto

Orientación a objetos

Un proyecto existe para lograr un resultado, y ese resultado se define mediante objetivos

- Los proyectos están dirigidos a resultados
- Los objetivos guían el proyecto
- No deben ser ambiguos
- Deben ser alcanzables
- necesidad → requerimientos → objetos concretos

Duración limitada

- Un proyecto es temporal
- Una línea de producción no es un proyecto
- No es una operacion continua

Tareas interrelacionadas

Un proyecto está compuesto por actividades conectadas entre si

- No son tareas aisladas, depende una de otras y requieren coordinacion
- Hay que tener en cuenta la complejidad sistematica, cambiar una parte afecta a otras

Son unicos

No existen técnicas universales para todos los sistemas, porque cada software tiene características distintas

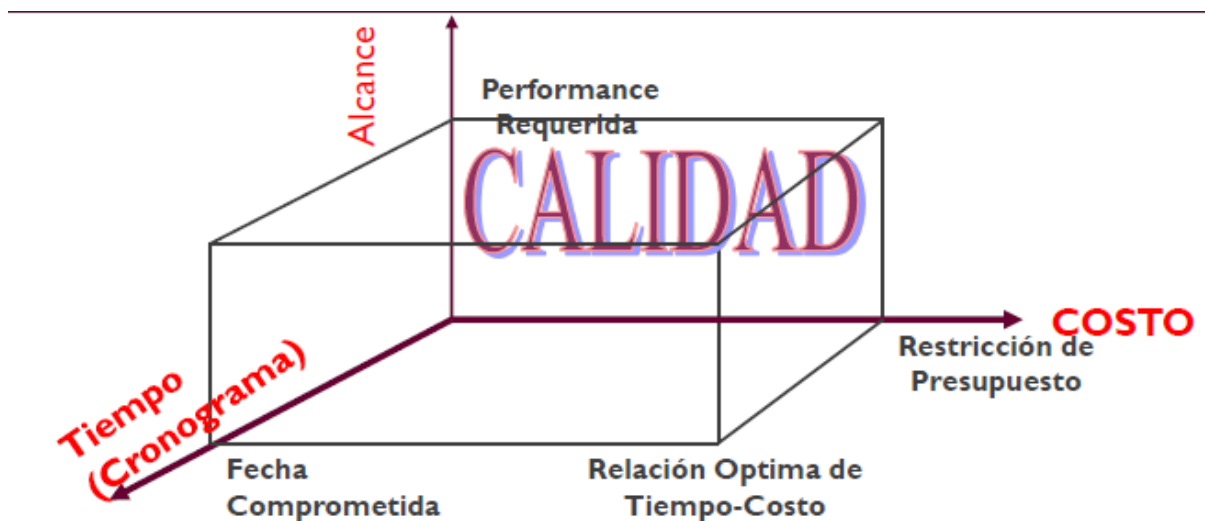
- Cada proyecto es irrepetible, por mas que sean similares cambia el contexto

Un proyecto se caracteriza por estar orientado a objetivos claramente definidos, tener una duración limitada en el tiempo, estar compuesto por tareas interrelacionadas que requieren coordinación de recursos, y generar un resultado único, lo que implica que cada proyecto presenta particularidades propias dentro del contexto en que se desarrolla.

Restricción triple

La restricción triple  La restricción triple establece que todo proyecto está condicionado por tres variables fundamentales: el alcance, el tiempo y el costo.

- Estas variables determinan que se quiere lograr, en cuanto tiempo se realizara y cuanto tiempo costara llevarlo a cabo
- La administración del proyecto consiste en gestionar estas tres dimensiones de manera equilibrada para cumplir con los requerimientos establecidos
- La gestión de proyectos implica planificar, organizar y controlar los recursos y actividades para alcanzar los objetivos del software



Alcance

Alcance ■ Hace referencia a aquello que el proyecto busca lograr, es decir, al conjunto de funcionalidades o resultados que se deben entregar.

- Define el nivel de desempeño o "performance requerida" del sistema
- El alcance se relaciona con los requerimientos del software ya que estos determinan que debe hacer el sistema

Tiempo

Tiempo ■ El tiempo representa la duración del proyecto y la fecha comprometida de entrega. Establece los plazos dentro de los cuales deben ejecutarse las actividades.

- Si el tiempo se reduce, será necesario disminuir el alcance o aumentar los recursos para poder cumplir con los objetivos.

- Pressman → Explica que la calendarización del proyecto es una de las tareas centrales de la gestión, ya que permite organizar las actividades y controlar su avance.
- Sommerville → Señala que los proyectos de software deben cumplir plazos definidos, aun cuando los requerimientos puedan cambiar.

Costos

Costos  El costo refiere a los recursos económicos necesarios para llevar adelante el proyecto, incluyendo personal, herramientas y tecnología.

- Un aumento en el costo puede permitir ampliar el alcance o reducir el tiempo, mientras que una reducción del presupuesto obliga a ajustar alguna de las otras variables.
- Pressman → El costo está directamente relacionado con la asignación de recursos y la estimación del esfuerzo requerido.
- Sommerville → Destaca que los proyectos deben ajustarse a presupuestos definidos, lo que condiciona las decisiones de desarrollo.

Relación entre las 3 variables



No pueden estar los 3 factores optimizados por igual entonces SIEMPRE se generan compensaciones
En base al contexto el líder del proyecto va a tomar decisiones estratégicas en las que definirá que optimizar y que no

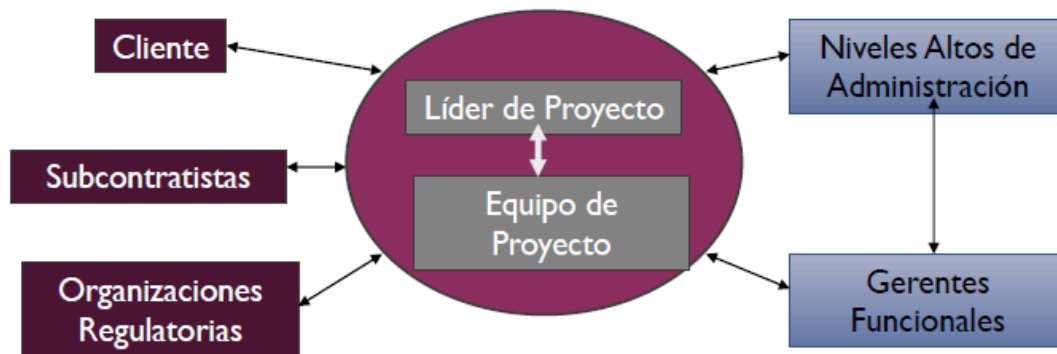
Las tres variables de la restricción triple se encuentran interrelacionadas entonces cualquier cambio en una de ellas genera un impacto directo en las demás.

- Si se amplía el alcance del proyecto, será necesario aumentar el tiempo de desarrollo o incrementar el costo.
- Si se reduce el tiempo de entrega, se deberá disminuir el alcance o aumentar los recursos para mantener la calidad.

Pressman plantea que la gestión del proyecto consiste en controlar estas variables de manera integrada

Sommerville destaca que los proyectos de software operan en entornos dinámicos donde este equilibrio debe ajustarse constantemente.

Equipo de proyecto



Equipo de proyecto  Un equipo de proyecto es un grupo de personas comprometidas en alcanzar un conjunto de objetivos, asumiendo responsabilidad compartida sobre los resultados

- **Cliente** → define requerimientos, el líder traduce esas necesidades al equipo y valida que se cumplan.
- **Niveles altos de administración** → fijan objetivos estratégicos, el líder reporta avances y alinea el proyecto con la visión de la organización.
- **Gerentes funcionales** → administran recursos (personas, áreas), el líder coordina con ellos para asignar y gestionar esos recursos en el proyecto.
- **Equipo de proyecto** → ejecuta el trabajo, el líder organiza, motiva, guía y controla las tareas.
- **Subcontratistas** → proveen servicios externos, el líder coordina entregas, tiempos y calidad.
- **Organizaciones regulatorias** → imponen normas, el líder asegura que el proyecto cumpla con requisitos legales y técnicos.

Plan de software

Plan de software  El plan de proyecto es el documento que organiza y guía la ejecución del proyecto, definiendo de manera clara qué se va a hacer, cómo se va a hacer, cuándo se va a hacer y quién será responsable de cada tarea.

- Sin planificación, el proyecto carece de dirección, lo que aumenta el riesgo de desvíos en tiempo, costo y alcance.

- El proyecto documenta:
 - **¿Qué se hace?** → define el alcance y las actividades del proyecto
 - **¿Cuándo se hace?** → establece el cronograma y los tiempos
 - **¿Cómo se hace?** → describe la metodología, herramientas y procesos
 - **¿Quién lo hace?** → asigna responsabilidades dentro del equipo

El plan permite reducir la incertidumbre, coordinar el trabajo del equipo y controlar el avance del proyecto.

Alcance



el proyecto se mide contra el plan, mientras que el producto se mide contra los requerimientos.

El alcance del proyecto → Define qué trabajo debe realizarse para entregar el producto final.

El alcance del producto → Incluye todas las funcionalidades que pueden formar parte del sistema, es decir, aquello que el software debe hacer.

- El cumplimiento del alcance del proyecto se evalúa en función del plan de proyecto, es decir, verificando si las tareas definidas se realizaron según lo planificado.
- El cumplimiento del alcance del producto se mide en relación con la especificación de requerimientos, evaluando si el sistema desarrollado cumple con las funcionalidades establecidas.

Estimaciones

Las estimaciones de software consisten en predecir de manera aproximada el tamaño, el esfuerzo, el tiempo y el costo necesarios para desarrollar un sistema.

Estas estimaciones son fundamentales para planificar el proyecto, asignar recursos y tomar decisiones.

- **Tamaño** → Magnitud del sistema a desarrollar
- **Esfuerzo** → El esfuerzo representa la cantidad de trabajo necesario para desarrollar el software

- Calendario → El calendario define cuánto tiempo llevará completar el proyecto.
- Costos → El costo se refiere al presupuesto necesario para desarrollar el software.
- Recursos críticos → Los recursos críticos son aquellos elementos indispensables para el desarrollo del proyecto, como el personal especializado, herramientas o infraestructura.

Todas estas variables están interrelacionadas. El tamaño influye en el esfuerzo, el esfuerzo determina el tiempo, y el tiempo junto con los recursos define el costo. Esto conecta directamente con la restricción triple, ya que cualquier cambio en una de estas variables impacta en las demás.

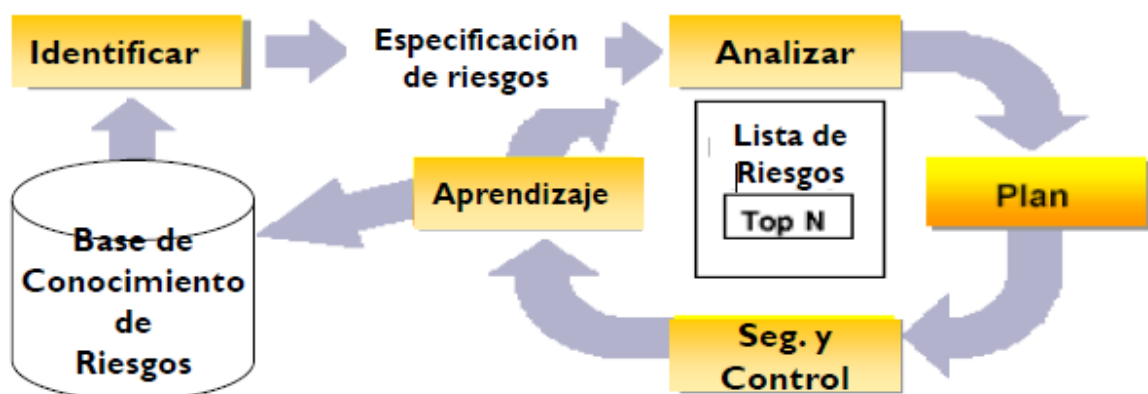
Gestión de riesgos

Riesgo ¶ Un riesgo en un proyecto de software es un evento potencial que aún no ha ocurrido, pero que podría afectar negativamente el desarrollo del proyecto o comprometer su éxito.

- Un problema esperando para suceder
- Un evento que podría comprometer el éxito del proyecto

Gestión de riesgos ¶ La gestión de riesgos consiste en identificar, analizar y controlar los riesgos a lo largo de todo el ciclo de vida del proyecto, con el objetivo de minimizar su impacto.

- Es una actividad continua que debe realizarse durante todas las fases del proyecto y no solo al inicio



- **Identificación** → detectar y reconocer los posibles riesgos que pueden afectar el proyecto.
- **Especificación de riesgos** → describir cada riesgo en detalle, incluyendo su causa y posibles consecuencias.
- **Análisis** → evaluar la probabilidad de ocurrencia y el impacto de cada riesgo para priorizarlos.
- **Planificación** → definir estrategias para prevenir o mitigar los riesgos identificados.
- **Seguimiento y control** → monitorear los riesgos durante todo el proyecto y verificar la efectividad de las acciones tomadas.
- **Aprendizaje** → registrar los riesgos y las experiencias para mejorar la gestión en futuros proyectos.

Métricas de software

Métricas de software 📌 Las métricas de software son medidas cuantitativas que permiten evaluar distintos aspectos del desarrollo de software, con el objetivo de mejorar la calidad, controlar el progreso del proyecto y facilitar la toma de decisiones

Tipos de métricas



Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software

Métricas de proceso

Se enfocan en cómo se desarrolla el software, es decir, evalúan la eficiencia y calidad del proceso utilizado

Métricas de proyecto

Las métricas de proyecto, en cambio, se centran en el seguimiento del estado del proyecto. Permiten controlar variables como el tiempo, el esfuerzo y el costo, facilitando la gestión del avance y la detección de desvíos respecto de lo planificado.

Métricas de producto

Evalúan las características del software desarrollado, como su tamaño, complejidad o calidad.

Métricas básicas de un proyecto

Tamaño del producto

El tamaño del producto mide la magnitud del software, ya sea en términos de líneas de código, funcionalidades o puntos de función.

- Es la base para estimar otras variables

Esfuerzo

El esfuerzo representa la cantidad de trabajo necesario para desarrollar el sistema

- Esta relacionado con el tamaño y la complejidad del software

Tiempo

El tiempo, o calendario, mide la duración del proyecto y permite evaluar el cumplimiento de los plazos establecidos.

Defectos

Miden la cantidad de errores detectados en el software, lo que permite evaluar la calidad del producto y la efectividad del proceso de desarrollo.

Las métricas permiten reducir la incertidumbre en los proyectos de software, ya que brindan información objetiva sobre el estado del proyecto y del producto. Además, facilitan la toma de decisiones, el control del progreso y la mejora continua del proceso de desarrollo.

Vínculo proceso-proyecto-producto

Relación conceptual

Esta relación es una de las ideas más importantes de la unidad:

- el **proceso** es la forma de trabajo;
- el **proyecto** es el esfuerzo temporal organizado;
- el **producto** es el resultado construido y luego evolucionado.

Confundir estos planos produce errores conceptuales. No es lo mismo hablar del método de trabajo que del esfuerzo de construcción ni del sistema

resultante.

Ejemplo

En la plataforma de cupos de traslados:

- el proyecto es el esfuerzo inicial para lanzar una primera versión;
- el proceso es la forma elegida para desarrollar esa versión;
- el producto es la plataforma como sistema vivo que seguirá evolucionando.

Unidad 2. Gestión de Producto, Requerimientos Ágiles, User Stories, Estimaciones Ágiles y Scrum

Gestión de Producto

La gestión de productos de software  Es una disciplina que se encarga de definir, desarrollar y evolucionar productos digitales con el objetivo de generar valor para los usuarios y para el negocio.

- Busca comprender las necesidades reales del usuario
- Validar continuamente que el producto construido responde a dichas necesidades.

El software debe entenderse como un producto vivo, que evoluciona a lo largo del tiempo y cuya calidad no depende únicamente de su funcionamiento técnico, sino de su capacidad para ser utilizado, adoptado y valorado por los usuarios.

Por qué se crean productos

Los productos de software se crean con múltiples objetivos, entre los cuales se destacan:

1. Satisfacer necesidades de los usuarios
2. Generar ingresos
3. Atraer usuarios
4. Materializar una visión estratégica.

Sin embargo, todos estos objetivos convergen en un punto común: la creación de valor.

El éxito de un producto no se mide por la cantidad de funcionalidades desarrolladas, sino por el impacto que tiene en los usuarios.

Valor vs desperdicio

Uno de los principios fundamentales del enfoque Lean es distinguir entre valor y desperdicio.

Valor 📌 Se considera valor todo aquello que contribuye directamente a satisfacer una necesidad del usuario

Desperdicio 📌 Cualquier actividad que no aporte a este objetivo

En el desarrollo de software, esto implica que construir funcionalidades que no serán utilizadas constituye una pérdida de recursos.

Por lo tanto, la productividad no se mide por la cantidad de código producido, sino por la capacidad de identificar y construir aquello que realmente importa.

El Problema del Sobre-desarrollo

Existe una tendencia natural en los equipos de desarrollo a agregar funcionalidades con la intención de mejorar el producto. Sin embargo, una gran proporción de estas funcionalidades no son utilizadas por los usuarios.

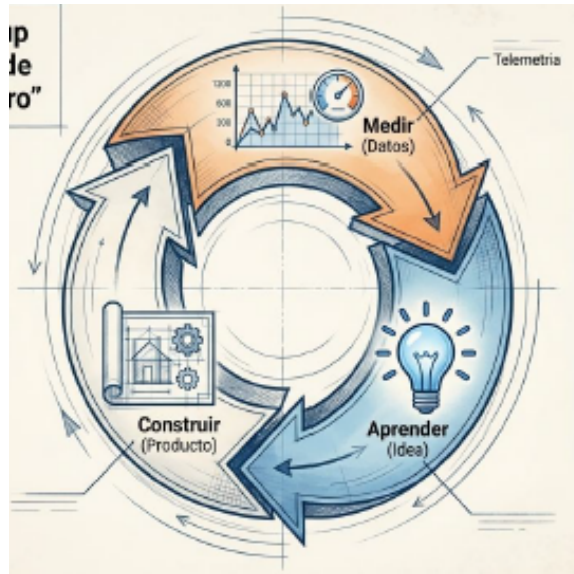
Este fenómeno genera una paradoja:



Cuanto más se construye sin validar, mayor es el riesgo de entregar menos valor

Por esta razón, la gestión de productos promueve enfoques que priorizan la validación temprana y la construcción incremental, evitando la sobrecarga de funcionalidades innecesarias.

Lean Startup y aprendizaje validado



Los productos se desarrollan basándose en hipótesis que deben ser validadas mediante la interacción con usuarios reales

El ciclo construir–medir–aprender constituye el núcleo de este enfoque. A través de este ciclo, los equipos pueden obtener aprendizaje validado, es decir, conocimiento basado en evidencia real sobre el comportamiento de los usuarios.

- Este enfoque permite reducir la incertidumbre y orientar el desarrollo hacia soluciones que realmente tengan sentido en el mercado.

Supuestos de salto de fe

Salto de Fe  Supuestos críticos de los cuales depende el éxito del producto, que representan las hipótesis mas riesgosas del modelo de negocio

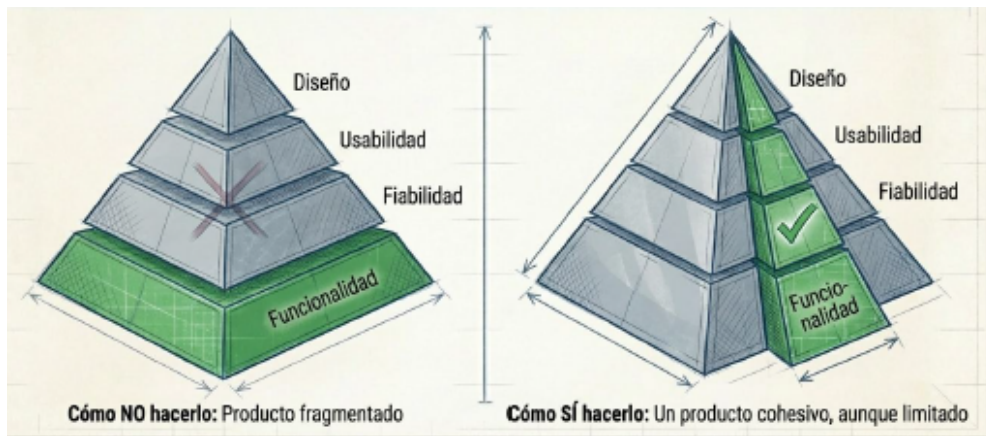
Dentro de estas hipótesis están:

- Hipótesis de valor  Evalúa si el producto satisface una necesidad real
- Hipótesis de crecimiento  Analiza cómo el producto puede expandirse en el mercado.

Validar estos supuestos es esencial para reducir el riesgo de fracaso.

Primera iteración

MVP



El MVP debe enfocarse en la propuesta de valor única (UVP) y ofrecer lo mínimo indispensable para ser viable

MVP El Producto Mínimo Viable es la versión más simple de un producto que permite obtener aprendizaje validado con el menor esfuerzo posible.

- Su objetivo no es ser un producto final, sino una herramienta para experimentar y aprender.
- El MVP puede adoptar distintas formas, lo importante es que se pueda observar el comportamiento real de los usuarios y obtener info para futuros feedback



Un MVP debe ser suficientemente valioso como para ser utilizado por los usuarios, pero lo suficientemente simple como para evitar el desperdicio de recursos.

Propuesta de Valor (UVP)

Propuesta de valor Es aquello que diferencia y le da sentido al producto en el mercado

- Es la hipótesis principal que debe ser validada a través del MVP

Una propuesta de valor clara permite orientar el desarrollo y evitar la dispersión en funcionalidades que no aportan valor real.

La Audacia del Cero y el Costo del Retraso



La solución de esto es la iteración continua a través del MVP ya que:
MVP **■** Es un experimento diseñado para confrontar la realidad lo antes posible

- Requiere simplificación y debe evitar la sobre construcción

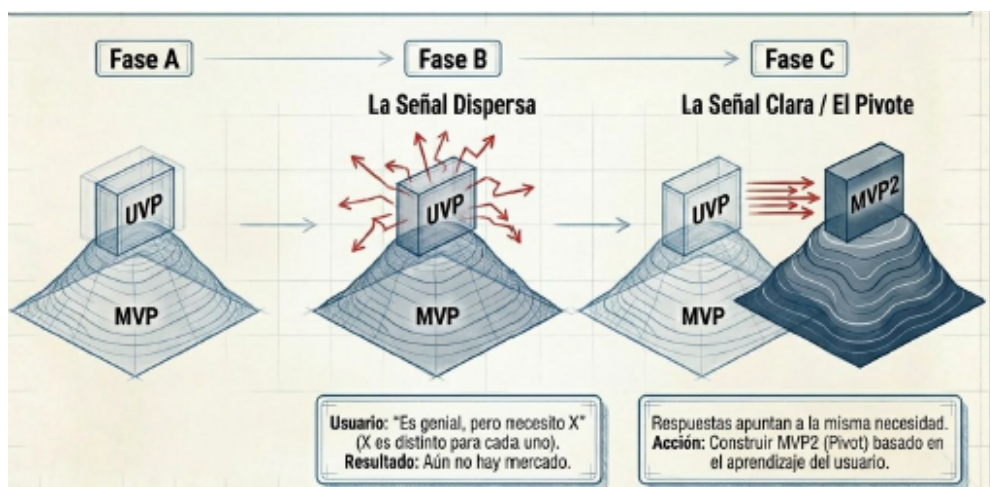
Audacia cero **■** Es la tendencia a retrasar el lanzamiento hasta alcanzar una versión "perfecta".

- Esto sucede porque la ausencia de usuarios o ingresos permite mantener expectativas idealizadas.

Retrasar el lanzamiento **■** Implica perder oportunidades de aprendizaje y aumentar el riesgo de construir un producto que no será utilizado.

Por lo tanto, lanzar temprano y aprender rápido resulta fundamental para el éxito.

Pívorot y Adaptación



Pívorot **■** Cambio estratégico basado en el aprendizaje adquirido

- Surge cuando los resultados obtenidos no confirman las hipótesis iniciales y entonces es necesario realizar un pivot
- El pivot permite ajustar el producto, redefinir el problema o modificar el mercado objetivo.

Este proceso es esencial para adaptarse a la realidad y avanzar hacia una solución más adecuada.

Ejemplo

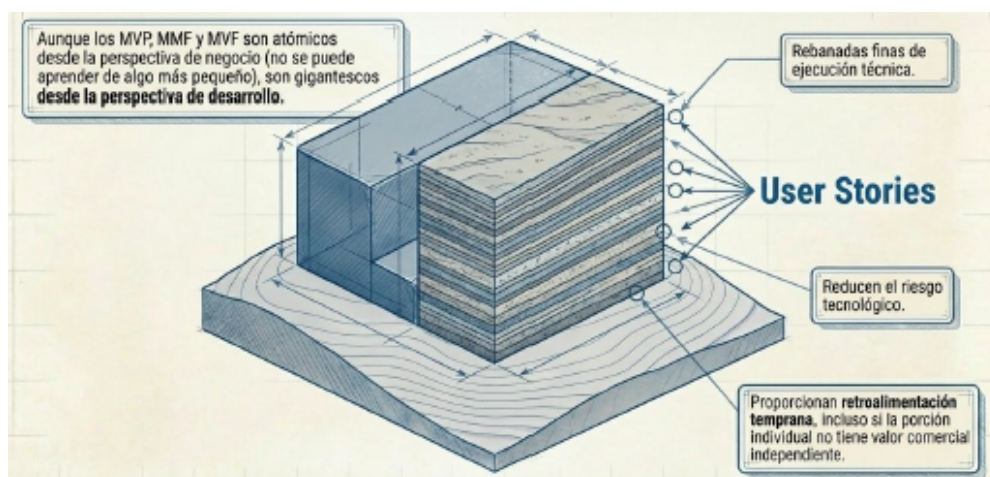
En la plataforma de cupos de traslados, una posible hipótesis de valor podría ser: "los usuarios están dispuestos a usar una plataforma centralizada para encontrar y coordinar traslados si pueden ver opciones relevantes de forma simple". Una hipótesis de crecimiento podría ser: "los transportistas independientes verán valor en publicar cupos si la plataforma les aporta visibilidad y ocupación".

Un MVP razonable podría incluir:

- registro básico de transportistas;
- publicación de cupos;
- búsqueda por origen, destino y fecha;
- un mecanismo simple de contacto o solicitud.

No haría falta, en una etapa inicial, construir reputación avanzada, segmentación compleja, reportes analíticos o automatizaciones sofisticadas si todavía no se validó el núcleo del valor.

Reducción del riesgo



Aunque el MVP, MVF y MMF son "mínimos" desde el punto de vista del negocio, pueden ser complejos desde el punto de vista técnico.

Para reducir el riesgo de implementación, se introduce el concepto de "corte geológico", que consiste en dividir el desarrollo en capas pequeñas llamadas user stories.

Estas pequeñas unidades permiten:

- reducir el riesgo técnico
- obtener feedback temprano
- iterar rápidamente

Ecosistema de los mínimos

Proceso de desarrollo del producto



1. Fase 1

El foco está en el aprendizaje mediante iteraciones rápidas cercanas al prototipo. Aquí se construyen MVPs.

2. Fase 2

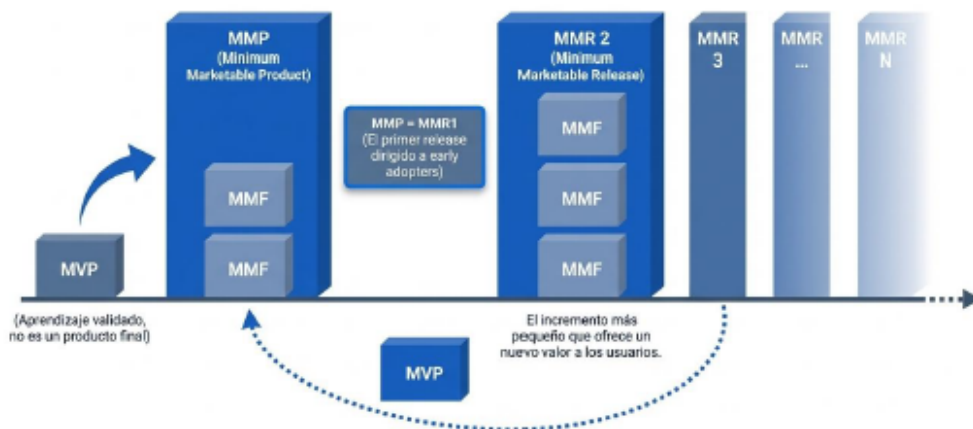
Se realiza el lanzamiento inicial del producto con funcionalidades esenciales, dando lugar al MMP.

3. Fase 3

El producto evoluciona mediante incrementos sucesivos, agregando pequeñas funcionalidades que aportan valor.



El producto no nace completo, sino que se construye progresivamente a través de ciclos de validación y mejora. El roadmap del producto debe ser adaptable para permitir cambios en función del aprendizaje aprendido.



El desarrollo de un producto puede entenderse como una secuencia evolutiva donde cada etapa cumple un objetivo distinto.

1. La primera etapa del desarrollo del producto es el MVP, se toma como un instrumento de aprendizaje que tiene como finalidad validar hipótesis con el menor esfuerzo posible
2. La segunda etapa del producto luego de obtener información de los usuarios con el MVP es el MMP que representa la primera versión del producto que puede ser lanzada al mercado con valor real para early adopters
3. A partir de acá, el producto evoluciona a través de releases incrementales (MMR) donde en cada uno se agrega pequeñas porciones de valor



Luego que se encuentra el encaje producto-mercado, el crecimiento se diseña combinando MMFs y MVFs

4. Para sostener el crecimiento y que el desarrollo del producto escale, dentro de cada releases incrementales se encuentran estos 2 mínimos que representan:

- MVF (Minimum Viable Features) 📌 Son funcionalidades pequeñas, experimentales que se desarrollan en contextos de alta incertidumbre
 - El objetivo es aprender y explorar nuevas oportunidades sin comprometer grandes costos
- MMF (Minimum Marketable Features) 📌 Representan funcionalidades que ya tienen certeza de valor
 - Son pequeñas unidades que pueden ser liberadas al mercado generando impacto real
 - Escalan lo que ya funcionan y mejoran el time of market

Experiencia del usuario y evolución del enfoque



Nos centraremos en un enfoque emocional donde el producto además de resolver un problema, nos generará satisfacción, diferenciación y fidelización



En la gestión moderna de productos, el MVP representa el punto de partida, ya que permite validar hipótesis y obtener aprendizaje con el menor esfuerzo posible.

Sin embargo, a medida que el producto madura y el mercado se vuelve más competitivo, la viabilidad deja de ser suficiente.

En este contexto surge el concepto de Producto Mínimo Adorable (MLP), que representa una evolución del MVP. Mientras que el MVP busca responder a la pregunta "¿esto funciona?", el MLP busca responder "¿esto genera conexión con el usuario?".

Producto Mínimo Adorable (MLP)



Mientras que el MVP se enfoca en validar hipótesis, el MLP busca generar una conexión emocional que favorezca la adopción y la fidelización.

El Producto Mínimo Adorable representa una evolución del concepto de MVP, incorporando la dimensión emocional del usuario.

- Este enfoque requiere un mayor nivel de diseño, atención a la experiencia de usuario y comprensión profunda de las necesidades del cliente.
- El MVP es una herramienta poderosa en contextos de alta incertidumbre pero en mercados donde ya hay competidores consolidados, un producto solo "viabke" podría no ser suficiente
 - Un producto que cumple su función pero no genera impacto pasa desapercibido
 - Por eso se deben crear productos no solo útiles sino MEMORABLES

Dimensiones del MLP

El MLP se construye sobre tres dimensiones fundamentales que transforman la relación con el usuario.

1. Reputación de marca

- Un producto debe transmitir que entiende y prioriza las necesidades del usuario, generando confianza desde el primer contacto.
- Esto es especialmente importante en empresas consolidadas, donde cada lanzamiento impacta directamente en la percepción de la marca.

2. Diferenciación

- En mercados saturados, no alcanza con cumplir lo básico.

- El producto debe destacarse, ofreciendo algo que capture la atención del usuario y lo motive a elegirlo frente a otras alternativas.

3. Feedback emocional superior

- A diferencia del MVP, que busca validar comportamiento, el MLP busca generar una conexión emocional que incentive el uso continuo y la lealtad.

Ejemplo

En la plataforma de cupos de traslados, pasar de MVP a MLP significaría que el sistema no solo permite encontrar cupos, sino que además transmite seguridad, claridad, cercanía y confianza. En un producto de intermediación, esto es clave porque gran parte del valor percibido depende de la confianza en el sistema y en la interacción que habilita.

Requerimientos ágiles

Procesos empíricos y requerimientos

Un **proceso empírico** es un proceso que se controla a través de la experiencia, la observación y la adaptación, en lugar de depender exclusivamente de una predicción inicial detallada.

- El enfoque empírico plantea que se haga una parte del trabajo, se observe lo que paso, se aprenda de eso y luego se corrija
- El enfoque empírico se centra mas en entornos complejos, cambiantes o inciertos
 - Por eso a los requerimientos no los vamos a tratar como algo cerrado, definitivo y detallado al inicio

Relación entre empírico y agilismo



Un proceso empírico **no es desorden**.

Significa tener una dinámica disciplinada, pero flexible: trabajar, observar resultados y mejorar continuamente.

1. No suponer que todo se sabe al inicio
2. Aprender con lo que se construye
3. Ajustar el rumbo

El **agilismo** adopta procesos empíricos porque el desarrollo de software ocurre en contextos de cambio e incertidumbre, donde el aprendizaje continuo es más valioso que la falsa ilusión de control total inicial.

- Esto por mas que parezca ordenado, es muy difícil llevarlo a cabo ya que los requerimientos cambian, el usuario descubre nuevas necesidades, entre mas cosas
- El agilismo lo que hace es trabajar en ciclos cortos, revisar lo hecho y luego hacer feedback
- El agilismo además de planificar, también consiste en inspeccionar y en adaptarse permanentemente

Agilismo

Ágil  Ideología con un conjunto definido de principios que guían el desarrollo del producto

- El agilismo funciona como un marco de pensamiento
- El agilismo propone usar **el proceso suficiente** para ordenar el trabajo, pero sin caer en burocracia innecesaria.
- La verdadera diferencia no está en la cantidad de documentos, sino en que la información se produce cuando aporta valor y no solo para cumplir un procedimiento.
- Los métodos ágiles están preparados para **ajustar el rumbo** a medida que el proyecto avanza.
- En ágil lo más importante no es seguir una secuencia rígida de pasos, sino que las personas trabajen bien juntas.

Valores del agilismo



Individuos e interacciones sobre procesos y herramientas

Lo más importante en un proyecto de software es que las personas se comuniquen bien, colaboren y tomen buenas decisiones.

- Los métodos ágiles están más orientados a la gente que al proceso.

Software funcionando sobre documentación extensiva

En ágil, el principal indicador es cuánto del sistema **realmente funciona**.

- El producto se valida viendo software real en acción, no solo leyendo documentos.

- Los métodos ágiles se apoyan en la entrega incremental y en mostrar resultados concretos

Colaborar con el cliente sobre negociación contractual

- El cliente o representante del negocio participa durante todo el proceso.
- El producto se entiende mejor a medida que avanza.
- En vez de encerrarse en un contrato rígido, se trabaja con **feedback continuo**, priorización y ajuste.

Responder al cambio sobre seguir un plan



Los metodos agiles son adaptables, no predictivos

Si cambia el negocio, el usuario aprende algo nuevo o aparece una restricción técnica, el equipo debe poder adaptarse.

Este valor conecta directamente con la idea de **procesos empíricos** que vimos antes: se avanza, se observa, se aprende y se corrige.

Principios del Manifiesto Agil

- 1 Nuestra mayor prioridad es **satisfacer al cliente**.
- 2 **Aceptar** que los requisitos cambien.
- 3 **Entregar** software funcional frecuentemente.
- 4 Los responsables de negocios, diseñadores y desarrolladores deben **trabajar juntos** día a día durante el proyecto.
- 5 Desarrollamos proyectos en torno a **individuos motivados**.
- 6 El método más eficiente de comunicar información es **conversaciones cara a cara**.
- 7 El **software funcionando** es la principal **medida de éxito**.
- 8 Los procesos ágiles promueven el **desarrollo sostenible**.
- 9 La **atención continua** a la **excelencia técnica** y al **buen diseño** mejor la Agilidad.
- 10 La **simplicidad** es esencial.
- 11 Las mejores arquitecturas, requisitos, y diseños emergen de **equipos auto-organizados**.
- 12 A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo y de acuerdo a esto **ajustan su comportamiento**.

1. Satisfacer al cliente

La prioridad es entregar algo que realmente le sirva al cliente y le aporte valor. No trabajar “porque sí”, sino para resolver su necesidad.

2. Aceptar cambios en los requisitos

En ágil, que cambien los requerimientos no se ve como un problema, sino como algo normal. El equipo debe poder adaptarse.

3. Entregar software funcional frecuentemente

No esperar hasta el final para mostrar resultados. Se entrega seguido, en partes útiles y funcionando.

4. Negocio y desarrollo trabajando juntos

Los responsables del negocio y el equipo técnico deben colaborar todos los días, no trabajar separados.

5. Trabajar con personas motivadas

Los proyectos salen mejor cuando el equipo está comprometido, tiene apoyo y confianza para hacer su trabajo.

6. La comunicación cara a cara es la mejor

Hablar directamente evita malos entendidos, acelera decisiones y mejora mucho la comprensión.

7. El software funcionando es la medida principal de éxito

El verdadero avance no se mide por documentos o promesas, sino por cuánto del sistema ya funciona.

8. Desarrollo sostenible

El trabajo debe poder mantenerse en el tiempo sin quemar al equipo. No sirve avanzar rápido hoy y destruir al equipo mañana.

9. Excelencia técnica y buen diseño

La calidad técnica importa. Un buen diseño y buenas prácticas ayudan a que el sistema pueda crecer y adaptarse mejor.

10. La simplicidad es esencial

Hay que hacer lo necesario, no complicar de más. En ágil se valora evitar trabajo innecesario.

11. Los mejores resultados surgen de equipos autoorganizados

Un buen equipo no necesita que le digan cada detalle. Puede organizarse, decidir y encontrar buenas soluciones.

12. Reflexionar y ajustar

El equipo debe parar cada tanto, revisar cómo está trabajando y mejorar. Esta es una base muy importante del agilismo.

Requerimientos ágiles

Se entienden como un camino que conecta una necesidad, problema u oportunidad con una solución entregable.

El foco inicial está en comprender el problema correcto y encontrar el camino más efectivo para resolverlo.

Eso cambia la pregunta inicial. En vez de pensar primero en pantallas o módulos, se piensa primero en:

- qué problema existe,
- qué valor se quiere generar,
- y cuál es la forma más efectiva de resolverlo.

Requerimientos en Agiles

La gestión ágil de requerimientos se puede resumir en tres ideas:

1. Usar el valor para construir el producto correcto.

No se trata de construir mucho, sino de construir aquello que tenga impacto.

2. Usar historias y modelos para mostrar qué construir.

Las representaciones son más livianas y conversacionales.

3. Determinar qué es solo lo suficiente.

Se busca el nivel de precisión necesario para avanzar, sin desperdicio documental.

Costo del BRUF

BRUF significa Big Requirements Up Front 📄 Significa definir grandes cantidades de requerimientos al inicio del proyecto.

- El problema además de documentación extensa es invertir tiempo y recursos en definir cosas que todavía pueden cambiar
- La agilidad prefiere descubrir parte del requerimiento a medida que el producto evoluciona.

Gestión ágil de requerimientos

Los requisitos cambiantes pueden convertirse en una ventaja competitiva si la organización es capaz de actuar sobre ellos.

En agilidad:

- Cada iteración implementa los requerimientos de mayor prioridad.
- Cada nuevo requerimiento puede agregarse y priorizarse.
- Los requerimientos pueden ser re-priorizados en cualquier momento.
- Los requerimientos también pueden eliminarse si dejan de aportar valor.

Cada iteración se concentra en lo que hoy tiene más valor. El cambio no se interpreta como una falla del proceso, sino como una característica natural del desarrollo de software.

Analice cuando lo necesite, no antes

El análisis detallado debe realizarse justo a tiempo, antes de comenzar el trabajo correspondiente, y no mucho antes.

Así se evita:

- analizar de más,
- congelar decisiones prematuramente,
- documentar detalles que luego cambian,
- generar inventario de requerimientos innecesarios.

Riqueza del canal de comunicación

El material destaca que la comunicación cara a cara es la más rica. Permite que fluya información verbal, gestual y contextual, con retroalimentación inmediata.

Tipos de requerimientos

1. Requerimiento de negocio
Expresa el objetivo de mayor nivel.
2. Requerimiento de usuario
Expresa qué necesita hacer el usuario.
3. Requerimiento funcional
Expresa qué debe hacer el sistema.
4. Requerimiento no funcional
Expresa restricciones o atributos del sistema.
5. Requerimiento de implementación
Expresa decisiones tecnológicas o de entorno.

Dominio del problema y dominio de la solución

- **Dominio del problema:** requerimientos de negocio y de usuario.
- **Dominio de la solución:** requerimientos de software.

La idea es comprender primero la necesidad antes de bajar rápidamente a la implementación.

Ejemplo

En la plataforma de cupos de traslados:

- requerimiento de negocio: aumentar visibilidad y ocupación de cupos;
- requerimiento de usuario: encontrar un traslado según fecha y destino;
- requerimiento funcional: permitir publicar y buscar cupos;
- requerimiento no funcional: responder en tiempos razonables y ofrecer disponibilidad alta;
- requerimiento de implementación: operar como sistema web/mobile con determinada infraestructura.

User Stories

Qué son

User Stories  Son expresiones breves de necesidad que funcionan como punto de partida para una conversación de valor.

- Buscan ofrecer una unidad liviana, comprensible y priorizable.
- Su fuerza no está solo en la frase escrita, sino en la conversación que la acompaña.

Product Backlog y Product Owner

El Product Owner prioriza las historias en el Product Backlog.

- Los ítems de mayor prioridad reciben mayor detalle.
- Los de menor prioridad permanecen más livianos.
- Cada iteración implementa primero lo más importante.
- Los ítems pueden agregarse, cambiar de prioridad o eliminarse.

Historias como porciones verticales

Las historias deben pensarse como cortes funcionales que atraviesan varias capas del sistema, por ejemplo:

- interfaz,
- lógica de negocio,
- base de datos.

No se busca organizar el trabajo solo por capas técnicas, sino por incrementos que ya entreguen valor funcional.

Partes de las User Stories

1. Tarjeta

Resume la historia.

2. Conversación

Aclara significado, alcance y detalles.

3. Confirmación

Define cómo se verificará que la historia fue implementada correctamente.

1. Estructura básica

Una forma típica es:

Como <rol>

quiero <necesidad o acción>

para <valor o beneficio>

Esta estructura obliga a pensar tres cosas importantes:

- quién necesita algo,
- qué necesita,
- y por qué eso tiene valor.

2. Modelado de roles

Escribir buenas historias depende mucho de comprender bien quién usa el sistema. Por eso se trabajan conceptos como rol, persona, personaje extremo o usuario representante.

- Rol de usuario: Tarjetas que describen características del usuario

- Las personas son perfiles ficticios pero realistas que permiten representar mejor a ciertos tipos de usuarios
- Personajes extremos Revelan necesidades que podrían pasar desapercibidas
- Existen los usuarios representantes que surgen cuando no hay acceso directo al usuario final

3. Criterios de aceptación

Criterios de aceptación ■ Definen los límites de una user story y ayudan a que todos compartan el mismo entendimiento sobre qué debe cumplirse para considerarla valiosa y terminada.

Para qué sirven

- Ayudan al Product Owner a expresar qué necesita.
- Ayudan al equipo a compartir una visión común.
- Ayudan a derivar pruebas.
- Ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad.

Cómo debe ser un buen criterio de aceptación

- Define una intención, no una solución.
- Es independiente de la implementación.
- Se mantiene en un nivel relativamente alto.
- No necesita describir todos los detalles posibles.

¿Dónde van los detalles?

Los detalles finos que surgen de las conversaciones pueden quedar en:

- documentación interna del equipo,
- pruebas de aceptación automatizadas.

4. Pruebas de aceptación

Pruebas de aceptación ■ Expresan en forma más concreta los detalles necesarios para validar una historia. Complementan a la user story y a sus

criterios de aceptación.

Relación con los criterios

Los criterios marcan el marco general. Las pruebas muestran casos concretos de éxito, error, borde o excepción.

Proceso de dos pasos

1. Identificarlas al dorso de la historia.
2. Diseñar las pruebas completas.

Ejemplo

La presentación agrega otra historia:

Como conductor quiero buscar un destino a partir de sus coordenadas para conocer el camino a recorrer para llegar al destino deseado.

Los criterios aclaran formato y orientación, y las pruebas validan casos correctos e incorrectos.

Ejemplo

Otra historia muestra un pago con tarjeta de crédito, con criterios como aceptar Visa, MasterCard y American Express, y pedir código de seguridad en compras mayores a cierto monto. Luego aparecen pruebas para tarjetas válidas, inválidas, vencidas o con datos faltantes.

Definition of Ready

Es la definición que indica cuándo una historia está suficientemente preparada para ser tomada por el equipo.

Definition of Done

Es la definición que indica cuándo una historia puede considerarse realmente terminada.

Aunque la diapositiva solo las menciona, conceptualmente sirven para alinear expectativas y evitar interpretaciones distintas sobre preparación y cierre del trabajo.

INVEST

Una buena historia debe ser:

- independiente: Debe ser independiente, de modo que pueda calendarizarse e implementarse en cualquier orden razonable.
- negociable: Debe expresar el qué y dejar espacio para conversar el cómo.
- valiosa: Debe aportar valor al cliente o al negocio.
- estimable: Debe poder estimarse para ayudar a priorizar.
- pequeña: Debe ser suficientemente pequeña como para consumirse dentro de una iteración.
- testeable: Debe poder demostrarse que fue implementada.

Spikes

Son historias especiales orientadas a reducir incertidumbre técnica o funcional.

- Se usan cuando todavía no hay suficiente claridad para comprometer desarrollo normal.
- Deben ser estimables, demostrables y aceptables.
- Deben reservarse para incertidumbres críticas o grandes.
- Deben usarse como excepción, no como regla.
- Salvo casos pequeños, conviene implementarlos en una iteración separada de las historias resultantes.

Ejemplo

Una User Story posible sería:

Como usuario, quiero buscar cupos disponibles por origen, destino y fecha, para encontrar una opción de traslado conveniente.

Criterios de aceptación posibles:

- solo deben mostrarse cupos activos;
- la búsqueda debe respetar origen, destino y fecha;
- el sistema debe informar cuando no existan resultados;
- la respuesta debe ocurrir dentro de un tiempo razonable.

Un spike técnico posible podría consistir en investigar cómo manejar búsquedas eficientes cuando la cantidad de publicaciones crezca.

Un spike funcional podría consistir en prototipar la experiencia de filtrado y comparar comprensión de usuarios.

Estimaciones tradicionales

- Las estimaciones ágiles no buscan eliminar la incertidumbre, sino hacerla visible y manejable.
- Deben entenderse como aproximaciones razonadas que sirven para decidir, priorizar y planificar.
- No son promesas exactas.

Para qué estimamos

Permiten anticipar si una funcionalidad parece razonable para una iteración

A medida que estimamos y el proyecto avanza, se vuelve más confiable porque el proyecto se apoya en evidencia concreta

Además esto nos ayuda para:

- Habilitar conversaciones más realistas sobre alcance, esfuerzo, costo, prioridad y factibilidad
- Revisar si el trabajo imaginado guarda relación con la capacidad real del equipo
- Permite detectar temprano si el trabajo planificado es demasiado grande o incierto.

Error de estimación



La estimación es mayor al inicio y se reduce gradualmente a medida que el proyecto avanza

Los errores de estimación surgen de la propia naturaleza incierta del desarrollo de software

Por eso a veces el problema no es "estimar mal", sino también, estimar en contextos donde todavía no existe toda la información necesaria

Técnicas fundamentales de estimación

Una técnica básica de estimación consiste en contar o descomponer el trabajo en partes observables y comparables.

Contar  Significa transformar un alcance abstracto en elementos discutibles: funcionalidades, historias, componentes o partes del producto.

- Se divide el problema y se observan mejor las partes
- Se lleva a cabo una estimación o comparación relativa

Métodos de estimación

Datos históricos

Permiten calibrar expectativas con base en experiencias previas comparables.

- Se reduce el sesgo puramente intuitivo
- Si un equipo sabe cuanto tardó en iteraciones pasadas, puede construir expectativas más realistas
- El pasado no debe ser una copia sino una referencia

Juicio experto puro

Aprovecha la intuición y experiencia de una persona competente.

Juicio experto estructurado

Ordena el juicio con pasos, criterios y revisión de cobertura.

- Trabaja con tareas de tamaño razonable, revisa cobertura, contempla escenarios optimistas, habituales y pesimistas
- Esto hace que el valor del experto lo vuelva más transferible y más robusto

Wideband Delphi

Integra múltiples expertos, comparación de estimaciones, discusión de supuestos y convergencia progresiva.

- Expertos estiman y comparan resultados hasta alcanzar una convergencia razonable

- Enriquece la comprensión del trabajo ya que las diferencias hacen que los supuestos y riesgos no pasen por alto

Estimaciones ágiles

Las estimaciones ágiles se basan en tamaño relativo, no en tiempo absoluto.

- Se emplean usando medida relativa de tamaño
- Esto es así porque las personas suelen comparar mejor que estimar de manera absoluta
- Se pregunta → *¿qué tan grande es esta historia en relación con otras?*
 - Esto favorece conversaciones más útiles y menos condicionadas por diferencias individuales
- En base al tamaño relativo, se proyecta cuando puede entrar en una iteración

Estimación relativa

La estimación relativa compara historias entre sí en lugar de asignarles un tiempo exacto desde el inicio

- Las personas son más consistentes cuando comparan que cuando intentan calcular valores absolutos.
- Mejora la discusión grupal

Tamaño de una historia

Tamaño 📏 Representa cuánto “pesa” una historia en términos de trabajo, complejidad e incertidumbre.

- Incluye cuánto trabajo requiere y cuánta duda o riesgo contiene

Tamaño vs esfuerzo

Tamaño 📏 Expresa la magnitud relativa de una historia

Esfuerzo 📏 El esfuerzo depende además de quién la realiza, cuánto conoce el dominio, qué experiencia tiene y qué obstáculos encuentra.

- Si estimamos en tiempo, se mezclan estos factores demasiado pronto

- Por eso la estimacion agil propone separar primero el tamaño y despues ver el esfuerzo o duracion

Escalas para estimar tamaño

- Talles de remera: S, M, L, XL.
- Escalas numéricas.
- Serie exponencial.
- Fibonacci.

Story points

Story point ■ Unidad relativa de tamaño definida por el equipo

- Son una medida relativa que combina complejidad, riesgo, duda y trabajo requerido
- Su valor es interno al equipo
- Comparan historias, las priorizan y luego en base a la velocidad, se proyecta cuanto trabajo puede completar el equipo en futuras iteraciones

Velocidad

Velocidad ■ Mide cuanto tamaño real completa un equipo por iteracion

- La velocidad surge de sumar los story points
- Esta metrica sirve para proyectar y ajustar planes futuros con base empira

Planning Poker

Es una tecnica grupal de estimacion relativa

Esto evita que una sola voz condicione al resto desde el comienzo y mejora la calidad de la conversacion

- Participan quienes van a desarrollar el trabajo.
- Se necesita una historia base o de referencia.
- Las cartas se muestran simultáneamente.
- Si hay diferencias, se discuten y se vuelve a votar.
- La comparación se hace contra historias ya entendidas.

Ejemplo

En la plataforma de cupos de traslados, no tendría el mismo tamaño una historia como “buscar cupos por fecha y destino” que otra como “implementar reputación y confianza avanzada entre usuarios y transportistas”.

La segunda probablemente combine más reglas, incertidumbre y casos excepcionales.

Por eso la estimación relativa ayuda a ordenar la complejidad antes de traducirla en planificación.

Framework Scrum

Qué es Scrum

Scrum es un marco de trabajo ágil basado en empirismo. No prescribe todas las técnicas del desarrollo, pero establece una estructura mínima de transparencia, inspección y adaptación para gestionar trabajo complejo.

Roles

Product Owner

Ordena y prioriza el Product Backlog buscando maximizar valor.

Scrum Master

Cuida el marco Scrum, remueve impedimentos y facilita mejora.

Developers

Construyen el incremento de producto.

Artefactos

Product Backlog

Lista dinámica y priorizada de trabajo.

Sprint Backlog

Conjunto de ítems seleccionados para el sprint y plan de ejecución.

Incremento

Resultado integrado, usable y coherente al final del sprint.

Eventos

Sprint

Contenedor temporal estable del trabajo.

Sprint Planning

Decide qué se hará y cómo se encarará.

Daily Scrum

Sincroniza al equipo diariamente.

Sprint Review

Permite inspeccionar el incremento con stakeholders.

Sprint Retrospective

Permite reflexionar sobre la forma de trabajo y mejorarla.

Relación con requerimientos y estimaciones

Scrum se integra naturalmente con requerimientos ágiles y estimación relativa:

- el backlog expresa valor en forma dinámica;
- las User Stories sirven como unidad frecuente de trabajo;
- la estimación ayuda a priorizar y planificar sprints;
- la review y la retrospective hacen operativa la lógica empírica.

Aplicación al ejemplo integrador

Un primer Sprint podría enfocarse en el mínimo núcleo del marketplace:

- alta de transportista;
- publicación de cupo;
- búsqueda básica;
- consulta de detalle.

En la Review se validaría si ese flujo ya permite comprobar la hipótesis central del producto. En la Retrospective se ajustaría la forma de trabajo del equipo.

Unidad 3. Gestión de Configuración de Software (SCM)

Gestión de Configuración de Software (SCM) → Es una disciplina de soporte que acompaña al proyecto durante todo su ciclo de vida.

Configuración → conjunto de artefactos y sus versiones en un momento determinado.

Integridad del producto → estado en el que el software conserva coherencia, trazabilidad y correspondencia con lo aprobado.

- Su función principal es mantener el orden en la evolución del software, evitando que los cambios generen pérdida de trazabilidad, inconsistencias entre artefactos, retrabajo o versiones imposibles de reconstruir.
- Desde una mirada práctica, SCM no se limita a “guardar código” o a usar Git.
 - En realidad, organiza y controla la evolución de todos los productos del proyecto: requerimientos, diseño, código fuente, ejecutables, casos de prueba, documentación, scripts de construcción, instaladores, manuales y registros del proyecto.

El software en contexto

SCM parte de una visión amplia del software. El software incluye:

- programas;
- procedimientos;
- reglas;
- documentación;
- datos.

Por eso, cuando cambia el software, no cambia solo el código: también pueden cambiar requerimientos, arquitectura, pruebas, manuales, scripts de despliegue y registros del proyecto.

Por qué cambia el software



SCM es una disciplina de soporte que trabaja como una actividad transversal a todo el proyecto

Los cambios pueden surgir por:

- nuevos requerimientos del negocio,
- modificaciones en productos asociados,
- cambios de prioridades organizacionales,
- restricciones presupuestarias,
- defectos detectados,
- oportunidades de mejora.

El cambio no es una excepción, sino una condición normal del desarrollo. El problema real no es que el software cambie, sino que cambie sin control.

Cuando eso ocurre, el equipo pierde visibilidad, se debilita la trazabilidad y se vuelve difícil reconstruir el estado real del producto en un punto del tiempo.e.

Relación con otras disciplinas

SCM se relaciona estrechamente con:

- aseguramiento de calidad;
- pruebas;
- gestión del proyecto;
- control de calidad de producto y proceso.

Esto se debe a que todas esas disciplinas necesitan saber **qué versión existe, qué fue aprobado, qué cambió y qué se está evaluando realmente.**

Problemas sin un buen SCM

- pérdida de componentes,
- pérdida de cambios,
- falta de sincronía entre fuente, objeto y ejecutable,
- regresión de fallas,

- doble mantenimiento,
- superposición de cambios,
- cambios no validados.

Estos problemas no son meramente técnicos.

En realidad, expresan fallas de coordinación y de control de información compartida. Un proyecto puede tener buenos programadores y aun así fracasar si no puede reconstruir una versión, saber qué cambió o demostrar qué fue aprobado.

- **Regresión de fallas** → Reaparición de errores que ya habían sido corregidos.
- **Superposición de cambios** → situación en la que varios cambios impactan sobre los mismos artefactos y generan conflicto.
- **Sincronía fuente-objeto-ejecutable** → correspondencia exacta entre código fuente, artefactos compilados y versión entregada.

Conceptos clave de SCM

1. . Ítem de Configuración de Software (SCI)

Ítem de configuración de Software → Un **ítem de configuración** es cada artefacto del producto o del proyecto que necesita control, seguimiento y conocimiento de su evolución.

2. Versión

Version → Es la forma particular que toma un artefacto en un instante o contexto dado.

El control de versiones se centra en la evolución de cada ítem por separado. Permite saber qué estado tuvo un artefacto y cómo fue cambiando.

3. Variante

Variante → Una variante es una versión que evoluciona por separado y representa una configuración alternativa.

4. Configuración del software

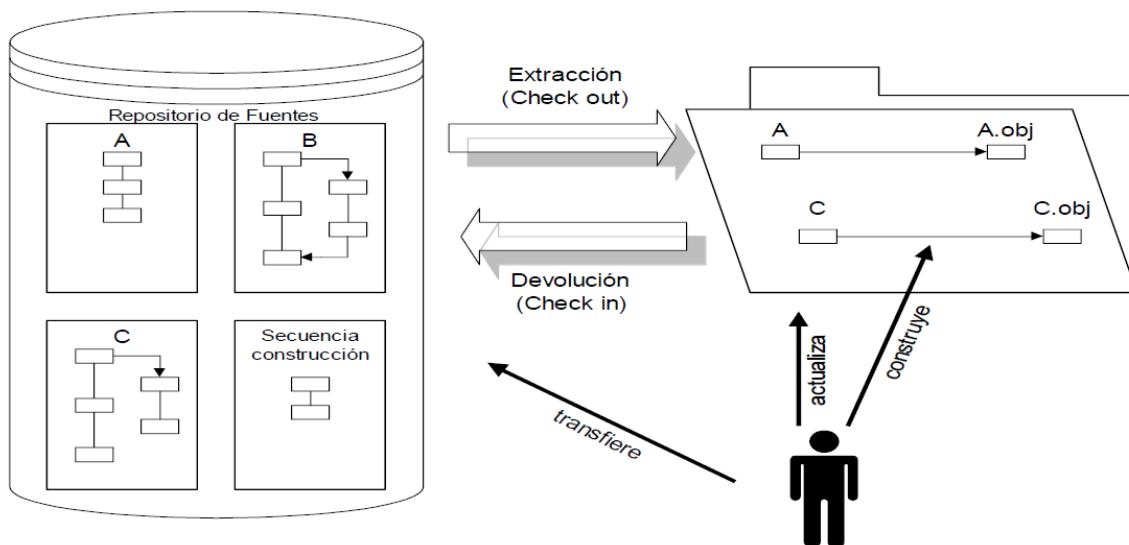
Configuración del software → Es el conjunto de ítems de configuración con sus correspondientes versiones en un momento determinado.

Ejemplos típicos

- producto para diferentes sistemas operativos,
- versiones con funcionalidades opcionales,

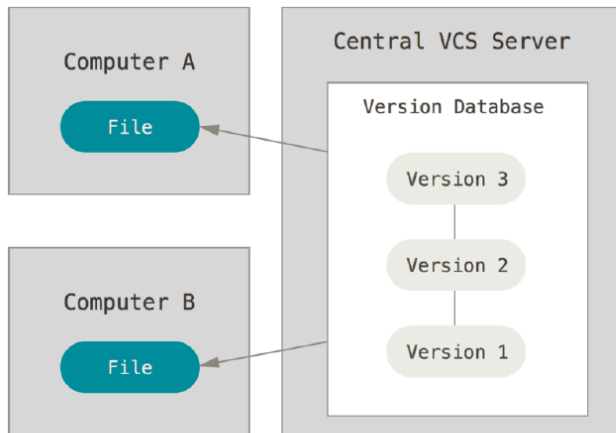
5. Repositorio

Repositorio → El repositorio es el espacio donde se almacenan los ítems de configuración, su historia, atributos y relaciones.



- conservar versiones,
- registrar evolución,
- facilitar recuperación,
- soportar análisis de impacto,
- servir de base para builds, auditorías y liberaciones.

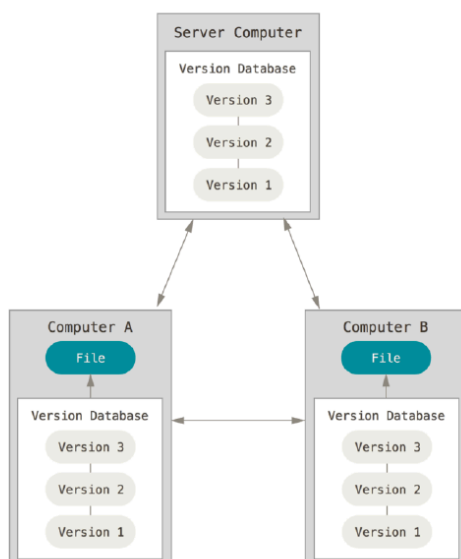
Centralizado



- ❖ Un servidor contiene todos los archivos con sus versiones.
- ❖ Los administradores tiene mayor control sobre el repositorio.
- ❖ Falla el servidor y "estamos al horno".

- un servidor concentra todas las versiones
- facilita control administrativo
- tiene dependencia fuerte del servidor

Descentralizado



- ❖ Cada cliente tiene una copia **exactamente** igual del repositorio completo.
- ❖ Si un servidor falla sólo es cuestión de "copiar y pegar".
- ❖ Posibilita otros workflows no disponibles en el modelo centralizado.

- cada cliente posee una copia completa del repositorio
- mejora resiliencia
- habilita workflows más flexibles

Línea base

Línea base → Una línea base es una configuración revisada formalmente y aceptada como referencia para el desarrollo posterior. Solo puede cambiar mediante un procedimiento formal.

¿Para qué sirve?

- fijar puntos de estabilidad,
- permitir volver atrás en el tiempo,
- reproducir estados del proyecto,
- comparar evolución,
- auditar cambios.

Tipos habituales

- línea base de especificación,
- línea base de diseño,
- línea base de producto.

Ramas

Las ramas permiten bifurcar el desarrollo a partir de una rama principal.

- experimentar,
- desarrollar una funcionalidad aislada,
- mantener una versión estable,
- corregir errores sin alterar la línea principal.

Actividades fundamentales de SCM

Identificación de ítems



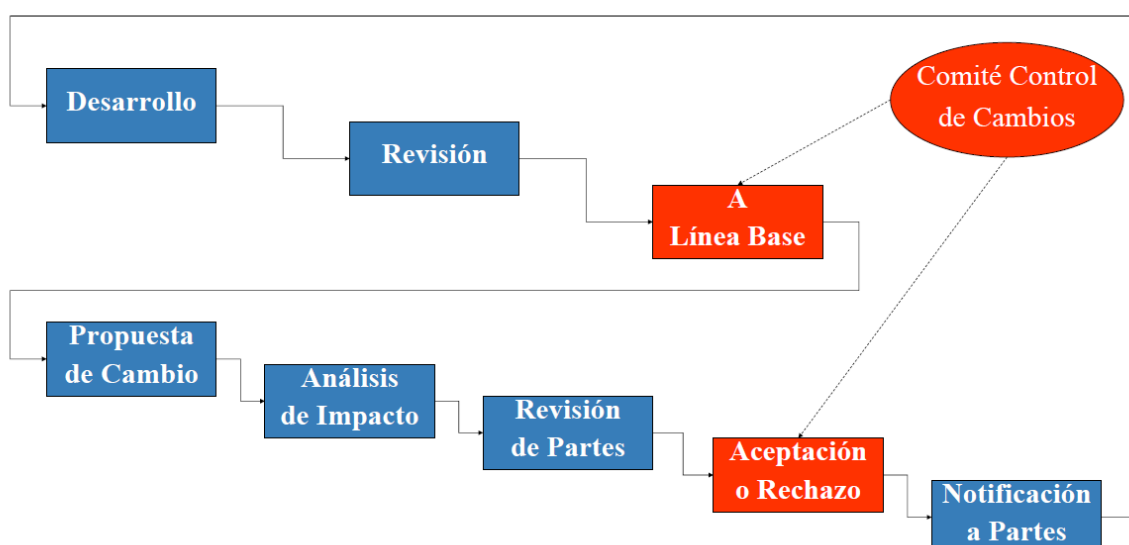
Consiste en definir qué se controlará y cómo se lo identificará.

Incluye

- identificación unívoca
- reglas de nombrado
- estructura del repositorio
- ubicación de cada ítem
- relación entre producto, proyecto e iteración

No se puede controlar lo que no está claramente definido. La identificación es la base del resto de las actividades de SCM

Control de cambios



Es el procedimiento formal que permite solicitar, evaluar, aprobar, rechazar, implementar y registrar cambios sobre ítems bajo línea base.

Comité de Control de Cambios

Está formado por representantes de las áreas involucradas.

Puede incluir

- análisis
- diseño
- implementación
- testing
- gestión
- interesados relevantes

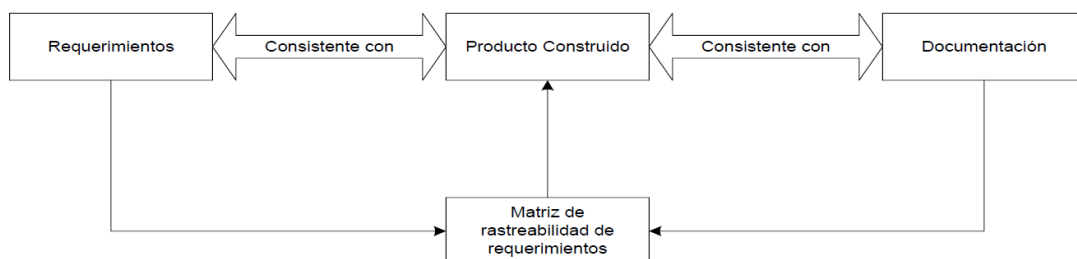
Función del comité

- evaluar solicitudes,
- revisar impacto,
- decidir aprobación o rechazo,
- priorizar cambios,
- asegurar coherencia del producto.

Auditorías de configuración

Auditoría Funcional de Configuración

Auditoría Física de Configuración



Verifican que el producto controlado coincide con lo especificado y que la configuración aprobada se ha implementado realmente.

Auditoría física (PCA)

Comprueba que lo indicado en la línea base realmente exista.

Auditoría funcional (FCA)

Comprueba que la funcionalidad y performance sean consistentes con la especificación de requerimientos.

Relación con verificación y validación

- **Verificación** → comprueba que el producto cumple con lo definido.
- **Validación** → comprueba que el producto resuelve correctamente el problema del usuario..

Informes de estado

Registran y comunican evolución de ítems y estado de los cambios. Permiten responder preguntas como: qué versión implementa cierta mejora, si un cambio fue aprobado o rechazado, y qué diferencias hay entre dos versiones.

Plan de SCM

Plan de SCM → documento que define políticas, procesos, roles y herramientas para la gestión de configuración.

Política de versionado → criterio adoptado para numerar y distinguir versiones.

SCM también se planifica. No debe improvisarse cuando el proyecto ya está en crisis.

Un plan de SCM debería incluir

- reglas de nombrado de ítems
- herramientas a utilizar
- roles y comité
- procedimiento formal de cambios
- plantillas y formularios
- procesos de auditoría
- reglas de versionado
- política de ramas y merges

- criterios de release

Planificar SCM implica definir cómo se controlará la evolución antes de que aparezca el caos.

SCM y desarrollo ágil

En enfoques ágiles, SCM no desaparece. Cambia de forma.

- servir a los practicantes;
- coordinar y no obstaculizar;
- responder al cambio;
- automatizar lo más posible;
- reducir desperdicio;
- ofrecer feedback continuo.

Esto encaja con integración continua, builds frecuentes, pipelines automáticos y trazabilidad liviana pero efectiva.

Ejemplo

En la plataforma de cupos de traslados, podrían considerarse ítems de configuración:

- historias priorizadas del backlog;
- criterios de aceptación;
- modelos de datos;
- arquitectura y decisiones técnicas;
- código fuente;
- scripts de base de datos;
- pruebas automatizadas;
- manuales operativos;
- documentación de despliegue.

Si se agrega, por ejemplo, una nueva funcionalidad de filtros avanzados de búsqueda, el cambio no impacta solo en el código. Puede modificar requerimientos, historias, pruebas, documentación y eventualmente decisiones

de arquitectura. Ahí se ve claramente por qué SCM es una condición de madurez: permite que el software evolucione sin perder coherencia.